Contents

Sur Protocol 1
Abstract . . . 1
1. Introduction . . . 2
2. Design Philosophy and Related Work . . . 3
3. System Architecture . . . 4
4. Identity Model . . . 6
5. Data Types and the Provenance Model . . . 7
6. Proof Generation: the Attestation Circuit . . . 8
7. Verification . . . 11
8. Proof Aggregation and Settlement . . . 12
9. The Post-Quantum Migration Path . . . 13
10. Security and Privacy Analysis . . . 14
11. Tokenomics . . . 14
12. Staged Delivery Plan and Current Status . . . 15
13. Honest Limits . . . 16
14. Conclusion . . . 17
15. References . . . 18

Sur Protocol

Cryptographic Provenance for Human-Created Content in the Age of Synthetic Media

A Technical Whitepaper

Version 3.0 — 2026

Abstract

Generative AI has reached the point where text, voice, images, and video produced by a machine are, on casual inspection, indistinguishable from content produced by a human being. Detection-based approaches — statistical classifiers that try to spot the fingerprints of a language model after the fact — are structurally losing this race: every improvement in detection becomes a training signal for the next generation of generator, and published studies already show unacceptable false-positive and false-negative rates on real text. **Sur Protocol takes the opposite approach.** Instead of trying to detect what is synthetic, Sur proves what is genuinely human, at the moment of creation, using the sensors and secure hardware already present in a personal device.

Sur is a three-layer system. On the device, a zero-knowledge circuit (gnark Groth16 over BN254) proves that a piece of text was produced by a human typing on a registered device, under physically-grounded behavioral constraints, without revealing the keystrokes, the device's identity, or the user's identity. A Cosmos SDK application chain, **Sur Chain**, verifies these proofs, maintains a privacy-preserving identity registry, and aggregates accepted attestations into a compact Merkle commitment. That commitment is designed to settle to public blockchains — an Ethereum contract is implemented and tested today, with Starknet and Solana planned next — so that no single party controls the record of what has been attested. Non-ZK content types (photos, video, audio,

declared/AI-authored text) are handled through a lighter device-signed declaration scheme with an explicit, honestly-labeled provenance class, so the system never overstates what it has actually proven.

This document describes the full pipeline — data types, cryptographic protocols, chain architecture, proof generation and verification, aggregation and multi-chain settlement, the post-quantum migration path, and the token economics that secure the network — as they are implemented today, with an explicit accounting of what remains open work.

---

1. Introduction

1.1 The problem

The question that started this project is simple to state and hard to answer: **can we ensure that content circulating on the Internet actually originates from the entity it claims to?** Every day, it becomes easier for a machine to reproduce a person's writing style, voice, or face convincingly enough to fool another human — and, increasingly, to fool automated systems too. The stakes are not abstract: a cloned voice can authorize a fraudulent wire transfer; a fabricated statement can be attributed to a public official; a synthetic video can be entered as false evidence. **Despite having a distinctive writing style, a voice, and a face, none of these will remain reliable proof of identity** once generative models close the remaining gap — from a private individual fighting a fraudulent claim to a head of state being made to appear to declare a war they never declared.

Detection-based defenses — classifiers trained to recognize "AI-like" patterns in text, audio, or video — face a structural problem: they are adversarial by construction. A detector that works today trains the next generation of generator to defeat it tomorrow. Independent evaluations of AI-text detectors report false-positive rates high enough to misclassify non-native English writers and neurodivergent writers as "AI-generated," and simultaneously miss a meaningful fraction of genuinely synthetic text — an unreliable foundation for anything with real consequences attached to it.

1.2 The analog-to-digital dilemma

Part of why detection is fundamentally hard is a property of digital media itself. The physical world is analog — continuous voltage, continuous light, continuous air pressure. Every microphone, camera, and touchscreen converts that continuous signal into discrete digital samples. By the Nyquist–Shannon sampling theorem, a signal of maximum frequency f_m must be sampled at $f_s \geq 2f_m$ to be reconstructed without aliasing, and every sample is then quantized to a finite number of bits (16-bit audio, for instance, maps continuous amplitude onto 65,536 discrete levels). This digitization step is lossy and one-directional. Once content is digital, a genuine recording and a synthetic one are the same *kind* of object — bits — and no purely digital analysis can recover the fact that one of them passed through a microphone and the other did not.

This is the core insight behind device-based provenance, and behind Sur: **the proof has to be attached at the moment of analog-to-digital conversion, on the device that did the converting, using hardware the device manufacturer — not Sur — already trusts.**

1.3 What Sur adds

Sur Protocol connects the physical act of creating data — typing, speaking, photographing — to a decentralized, publicly verifiable record, without a central authority deciding what counts as authentic and without requiring anyone to hand over the underlying content to prove it. Two design commitments run through every layer of the system:

- **Prove, don't detect.** Every claim Sur makes is backed by a specific cryptographic artifact generated at creation time — a zero-knowledge proof for human-typed text, a device signature for declared content — never a post-hoc statistical guess.
- **Say exactly what was proven, and nothing more.** Not all content types can be backed by the same strength of evidence today (see §5). Sur's data model makes the provenance *class* — human-typed, device-authored, AI-declared, quoted, imported — an explicit, queryable field, rather than collapsing everything into a single "verified" badge.

---

2. Design Philosophy and Related Work

2.1 Content provenance standards — C2PA

The Coalition for Content Provenance and Authenticity (C2PA) [1][2] is an open, widely-adopted standard for embedding tamper-evident provenance metadata — edit history, capture device, AI-disclosure flags — into media formats such as PNG, JPEG, and PDF. It is backed by Adobe, Microsoft, and a growing set of camera and AI-tooling vendors [3]. C2PA is complementary to Sur rather than competing with it: C2PA defines *how provenance metadata is packaged and signed for a given file format*; Sur adds *decentralized, device-attested proof of human origin* as one of the assertions that can back that metadata, and extends the same idea to typed text, which has no equivalent standard today.

2.2 Device attestation

Apple's App Attest / DeviceCheck services [4][5][6] let a server verify, via a hardware-backed attestation, that a request genuinely originates from an unmodified copy of a specific app running on genuine Apple hardware — without identifying the specific device or user. Qualcomm and other silicon vendors offer comparable chip-level attestation primitives. Sur uses App Attest as a *liveness and integrity* anchor (confirming the software making a proof hasn't been tampered with) layered underneath its own device-identity and commitment scheme (§4) — App Attest establishes that the *app* is genuine; Sur's own cryptography establishes *which pseudonymous device* produced a given attestation and prevents that device from being cloned into two identities.

2.3 Zero-knowledge provenance

Sur's use of zero-knowledge proofs to assert a property of private data without revealing the data follows a line of work that includes ZKPROV [7], which applies ZK techniques to dataset provenance for large language models, and the broader zero-knowledge proof literature [8]. The specific pattern Sur uses — commit to a private witness, prove a predicate over it in-circuit, publish only the proof and a small set of public inputs — is standard zk-SNARK practice [9]; what is novel is applying it to *keystroke dynamics* as the private witness, gated by physically-motivated behavioral

thresholds (§6.3), to produce a "typed by a human, on this specific registered device, during this specific session" claim.

2.4 Trust without consensus, and unclonable identity

Three pieces of prior work — by this project's own founder, predating Sur's implementation — shaped Sur's architecture directly enough to cite as primary design influences rather than background reading:

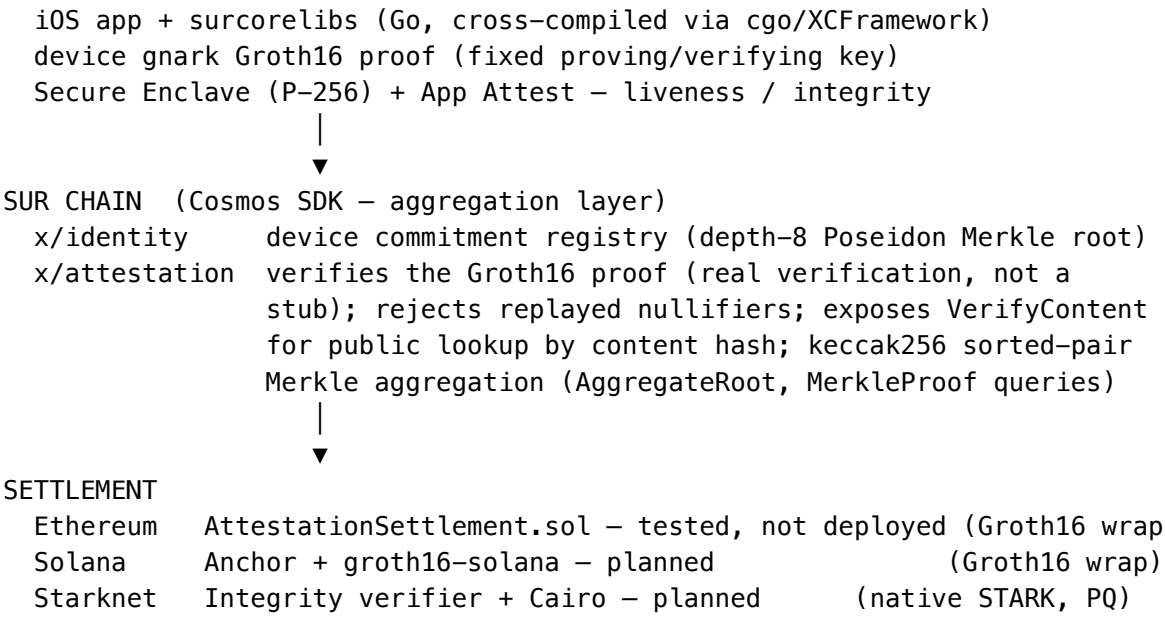
- **"Unreplicable Device IDs"** [10] argues that blockchain's dependence on live connectivity excludes anyone without reliable internet access, and proposes unclonable, unreplicable device identifiers — built from Physical Unclonable Functions or, more practically today, from hardware-backed key material — as an offline-capable alternative trust anchor for identity and transactions. Sur's device identity (§4) is a direct, shipping instance of this idea: a device-bound keypair standing in for a network consensus round as the thing two parties agree to trust.
- **"Bringing Trust to the Edge: Secure Execution Without Consensus"** [11] makes the case that not every trust decision needs a blockchain's consensus mechanism — a hardware Trusted Execution Environment can provide an equivalent, locally-verifiable execution guarantee for many purposes, while noting the real limitation: *"trust only exists when multiple parties agree on a shared truth,"* so a purely local guarantee still needs a way to be checked by someone else. Sur's architecture reflects exactly this division of labor: the Secure Enclave / App Attest layer gives *local* liveness and non-cloning guarantees on the device; the zero-knowledge proof and Sur Chain give the *externally checkable* guarantee that a hardware claim alone cannot provide.
- **"Commitment Schemes + Secure Enclave: A Powerful Combination"** [12] proposes that *"just as we trust the code running on a blockchain, we can also trust the code executed within a Secure Enclave,"* and that pairing a Secure Enclave with a cryptographic commitment scheme can reproduce blockchain-style behavioral guarantees on a single standalone device. Sur's device commitment — `Poseidon(pubkey_x, pubkey_y, blinding_factor)`, held in a Merkle tree the chain maintains — is precisely this pairing: the Secure Enclave anchors the key material; the Poseidon commitment is what makes that key material provable in zero knowledge later.
- **"EthSafari: ZK-Based KYC"** [13] draws a distinction Sur's own documentation leans on throughout: *data integrity* ("the accuracy, completeness, and consistency of data as it is stored") versus *computational integrity* ("the guarantee that the output of a computation is correct"), and argues that zero-knowledge proofs let a verifier confirm compliance "without ever revealing the private information itself." Sur's attestation circuit is a computational-integrity proof (§6) in exactly this sense: the chain never sees a keystroke, only a proof that a valid session existed.

---

3. System Architecture

Sur is organized into three layers: on-device proof generation, chain-level verification and aggregation, and multi-chain settlement.

```
DEVICE
```



Repositories. The system spans four repositories: `Sur/` (the iOS app and the `surcorelibs` Go proving library, cross-compiled to an XCFramework and linked into the app via a bridging header — not a Swift module, since a static-lib XCFramework's modulemap is not reliably importable); `SurChain/` (the Cosmos SDK application chain); `sur-evm-contracts/` (Ethereum settlement contracts); and `sur-protocol-docs/` (this document and the engineering roadmap it is drawn from).

Chain modules. Sur Chain is built with Cosmos SDK v0.53 and Ignite CLI, and runs the standard SDK module set — `auth`, `bank`, `staking`, `distribution`, `mint`, `slashing`, `gov`, `epochs`, `group`, `authz`, `consensus`, `params` — alongside four protocol-specific modules:

Module	Role	Status
`x/identity`	Device-commitment registry; maintains the depth-8 Poseidon Merkle root proofs are checked against	Live
`x/attestation`	Verifies attestation proofs/declarations; nullifier replay protection; content lookup; keccak Merkle aggregation for settlement	Live
`x/provenance`	Reserved for future provenance-metadata parameters	Scaffolded, no business logic yet
`x/surchain`	Reserved for chain-level protocol parameters	Scaffolded, no business logic yet

4. Identity Model

4.1 Why not usernames

An early version of Sur identified users by a chosen username, bound to their device commitments. This leaks exactly the kind of correlatable, persistent public handle the protocol exists to avoid creating: a single human-readable name links every attestation a person has ever made to their real identity, forever, in public chain history. Sur's current identity model replaces that with a **pseudonymous device identity** derived from key material the person already controls, and demotes the username to an optional, purely local display alias that never touches the chain.

4.2 Device identity derivation

Each device derives its own keypair without needing new hardware support:

```
device_seed       = HMAC-SHA256(user_private_key, device_uuid)
device_keypair    = secp256k1_keypair(device_seed)
device_id         = bech32("surdev", RIPEMD160(SHA256(compressed_device_pubkey)))
                      → e.g. surdev1qf3...
```

`user_private_key` is the user's own mnemonic-derived Sur account key; `device_uuid` is the platform's vendor identifier. The derivation is **recoverable** from the user's seed phrase (so losing a device doesn't strand a user's history) yet **device-scoped** (a factory reset produces a new UUID and therefore a new device identity — old attestations stay bound to the device that actually made them). The private half never leaves the iOS Keychain.

Sitting alongside the derived secp256k1 identity key is a hardware-backed P-256 keypair in the Secure Enclave, used for the device *commitment* (§4.3) and as a liveness/ anti-cloning control: because Secure Enclave keys cannot be extracted or imported, this control key gives a non-software-cloneable anchor for "this is a real, distinct piece of hardware" that App Attest can further vouch for at the OS level.

4.3 Device commitments and the identity Merkle tree

At registration, a device publishes a **commitment**, not its raw public key:

```
commitment = Poseidon(device_pubkey_x, device_pubkey_y, blinding_factor)
leaf       = Poseidon(commitment)                  // second-preimage guard
```

`x/identity` maintains a depth-8 Poseidon Merkle tree of these leaves (supporting up to 256 device commitments per identity; changing the depth requires a new trusted-setup ceremony, since it's a circuit-level constant). The attestation circuit later proves Merkle membership of a device's commitment against this root **without revealing which leaf, or the device's public key** — the chain only ever learns that *some* registered commitment produced this proof.

4.4 The privacy boundary that matters

Sur's privacy goal is specific, and worth stating precisely because "privacy" is otherwise a vague word: **attestations are, and are meant to be, linkable to a device's pseudonymous public identity** — that linkability is the provenance record itself; a verifier needs to be able to say "this content was attested by device `surdev1qf3...` three times before." What must **not** happen is a link

from that device identity back to a real-world person or account without the person's cooperation. The device id is not a human username; a human display name is optional, local-only metadata. Reaching from a device id to a person is not something the protocol currently offers as an automatic on-chain capability — it isn't wired up as a routine query — and the plainly-labeled residual gap is that the **transaction sender** (the account that pays gas to submit an attestation) is currently the user's own wallet account, which a sufficiently motivated chain-graph analyst could correlate across many attestations. The architecturally clean fix — a separately funded device-controlled account submitting its own transactions — is scoped but not yet built (§13).

4.5 AI agent identities

Autonomous AI agents get their own address space rather than being folded into device identities: `agent_id = bech32("surai", RIPEMD160(SHA256(compressed_agent_pubkey)))` (HRP `surai`, e.g. `surai1...`). An agent's declaration is **self-sovereign** — the agent's key signs the claim, and unlike a device declaration, no prior identity registration is required (the agent's key *is* its identity). Ownership is public by construction: the account that pays for and signs the surrounding transaction is visible on-chain alongside the agent's declaration, so "whose agent said this" is answerable without a separate registry. A richer registration flow (binding an agent id to a human-readable label and an accountable owner) is planned but not yet built.

---

5. Data Types and the Provenance Model

Not all content can be backed by the same strength of evidence, and Sur's data model says so explicitly rather than presenting a single undifferentiated "verified" badge. Every attestation carries an **origin** field:

Origin	Backed by	Claim it makes
`human_keyboard`	Zero-knowledge Groth16 proof (§6)	This exact text was typed by a human on this registered device, under physically-plausible typing dynamics
`device_authored`	Device signature (declaration)	This device produced/submitted this content; typing itself is not cryptographically verified
`ai_generated`	Device signature (declaration)	The device is declaring this content as AI-generated
`external_source` / `imported`	Device signature + optional citation	This content was quoted or imported from elsewhere; an optional citation names the source
`ai_agent`	Agent signature (self-sovereign)	This content was produced by a specific autonomous agent, whose key is its identity

Only `human_keyboard` is backed by the ZK circuit; every other origin is a **device- or agent-signed declaration** — a `secp256k1` signature over `SHA-256(content_hash || origin || citation || nullifier)` from a key that hashes to the claimed device or agent id. Declarations are honest by design: they assert what the signer *claims*, not what has been mathematically proven, and the system is explicit about the difference everywhere the distinction is surfaced (the public verification endpoint, the block explorer, the app itself).

5.1 The attestation record

On-chain, every accepted attestation is stored as:

```
message AttestationRecord {
  string username        = 1;  // the surdev1… / surai1… identifier
  bytes  content_hash    = 2;  // SHA-256(content)
  bytes  nullifier       = 3;  // anti-replay token, see §6.2
  bytes  commitment_root = 4;  // identity Merkle root (human_keyboard only)
  int64  timestamp       = 5;
  string origin          = 6;  // provenance class, see table above
  string citation        = 7;  // optional, external_source / imported only
}
```

submitted via `MsgSubmitAttestation`, which additionally carries `proof_bytes` (the Groth16 proof, `human_keyboard` only) or `device_pubkey` / `device_signature` (every declaration origin). The chain branches on `origin`: `human_keyboard` runs full Groth16 verification against the identity commitment root; every other origin runs declaration-signature verification instead. Both paths check the nullifier against the replay-protection set before accepting the message.

5.2 Public verification

Anyone — not just Sur users — can look up whether a specific piece of content has been attested, given only its SHA-256 hash:

```
GET /surprotocol/surchain/attestation/v1/verify/{content_hash_hex}
→ { found: bool, attestations: [ { username, origin, citation, timestamp, ... } ] }
```

This is the query the public verification page and the Sur browser keyboard use: paste or hash a message, and see every attestation on file for that exact content, each shown with its honest origin label rather than a blanket "verified" claim.

---

6. Proof Generation: the Attestation Circuit

6.1 Circuit overview

The `AttestationCircuit` is a gnark [14] Groth16 circuit over the BN254 curve. It proves, in zero knowledge, that:

1. The prover knows a device key whose Poseidon commitment is a member of the on-chain identity Merkle tree (depth 8).
2. The submitted nullifier equals `Poseidon(username_hash, session_counter, blinding_factor)`.

3. A set of behavioral statistics about the typing session fall within physically-motivated bounds (§6.3).
4. The two public halves of the content hash reconstruct the private, full 256-bit SHA-256 hash of the typed text.

```
Public inputs (5 BN254 field elements, fixed order — must match the on-chain
verifier's uint[5] layout exactly):
  0: username_hash        Poseidon over the identifier's UTF-8 bytes, packed
                           into 31-byte field limbs (computed off-circuit; the
                           chain recomputes it from the submitted identifier
                           and supplies it as this input, binding the proof)
  1: content_hash_lo       low 128 bits of SHA-256(message)
  2: content_hash_hi       high 128 bits of SHA-256(message)
  3: nullifier             anti-replay token (§6.2)
  4: commitment_root       identity Merkle root the proof was checked against

Private witnesses (zero-knowledge):
  device_pubkey_x/y       Secure Enclave P-256 coordinates, reduced mod the BN254
                           scalar field (P-256's field ≠ BN254's; values ≥ r are
                           taken mod r)
  session_counter          monotonically increasing per-device counter
  blinding_factor          32-byte secret from device registration (iOS Keychain)
  human_score, iki_min,
  iki_max, keystroke_count  behavioral statistics (§6.3)
  content_hash_priv         full 256-bit hash before splitting
  merkle_path[8],
  merkle_directions[8]      inclusion path for the device commitment
```

6.2 The nullifier and anti-replay

```
nullifier = Poseidon(username_hash, session_counter, blinding_factor)
```

The chain persists every nullifier it has ever accepted and rejects a resubmission outright — the on-chain `IsNullifierUsed` query and the `x/attestation` message handler both check this before anything else. Security here reduces to two assumptions: Poseidon behaves as a pseudorandom function over its input tuple (a heuristic, not a reduction-based guarantee — the same accepted tradeoff every ZK-friendly hash makes for in-circuit efficiency), and `blinding_factor` carries near-full BN254 scalar-field entropy and is known only to the device holder. Given those, no outside party can predict or grind a future nullifier, and no proof can be replayed to double-count a session — forging an alternate nullifier for an already-spent `(device, session_counter)` pair would require a Poseidon second-preimage.

6.3 Behavioral thresholds — what they prove, and what they do not

The circuit enforces range constraints on typing behavior as private-witness constraints — enforced inside the proof, invisible to the verifier, but load-bearing for whether a proof can be constructed at all: `human_score` (a device-computed behavioral quality score) in [50, 100]; `keystroke_count` ≥ 10 per attested session; and an inter-keystroke interval (IKI) window of `iki_min` ≥ 20 ms, `iki_max` ≤ 2000 ms. These are best understood as a **plausibility gate**, not a biometric identity proof: they

raise the cost of forging a session by requiring an adversary to match a joint numerical envelope, the same false-accept/false-reject standard used throughout the keystroke-dynamics literature [15] — never a certainty claim.

The 20 ms floor sits below essentially all recorded human bigram timings. Large-scale typing corpora — Dhakal et al., *"Observations on Typing from 136 Million Keystrokes,"* CHI 2018 [16], spanning roughly 168,000 participants — put average skilled-typist speed near 52 WPM, corresponding to mean inter-key intervals well above 100 ms; even at recorded extremes (sustained bursts near 200+ WPM) average IKIs land in the 35–40 ms range. Reaction-time floors from Card, Moran & Newell's Model Human Processor (*The Psychology of Human-Computer Interaction*, 1983) [17] place simple motor response in the 70–100 ms range. Sur's 20 ms cutoff is therefore a *deliberately permissive* absolute floor — biased toward accepting real-but-fast typists rather than rejecting them — and functions as a trip-wire against crude scripted/automated input rather than a tight anti-bot guarantee against a careful synthetic-timing generator.

The 2000 ms ceiling is on weaker ground and is best described honestly as an anti-idle/anti-stalled-session heuristic rather than a literature-backed value: published work on natural pause structure in transcription shows legitimate thought-boundary pauses routinely exceeding 1–2 seconds, so a hard per-session ceiling risks rejecting genuine sessions containing one natural pause, depending on exactly how the statistic is computed over the session. **The 10-keystroke minimum** is a statistical-sample-size floor (below which a per-session behavioral estimate is unstable), not a value drawn from a specific published source. **`human_score` in [50,100] is currently underspecified in this document**: the circuit range-checks the score but does not compute it, the scoring formula lives outside the circuit, and until that formula is documented and justified with the same rigor as the IKI bounds above, the 50-point floor should be treated as an open design question rather than an empirically-calibrated threshold.

What this buys, stated precisely: these bounds establish that a session's timing is *consistent with* unassisted human typing. They do not constitute strong biometric identification of a specific individual, and a sufficiently patient adversary with physical access to a device and its Keychain-held key material could still construct a passing session — the circuit's guarantee is about the plausibility of the input's shape, not possession of a particular finger.

6.4 Commitment hiding and Merkle domain separation

The device commitment `Poseidon(device_pubkey_x, device_pubkey_y, blinding_factor)` is hiding in the same sense a Pedersen commitment is hiding — because `blinding_factor` carries near-full field entropy independent of the key coordinates, the commitment is computationally indistinguishable from a uniform field element to anyone without it — except that this rests on Poseidon's assumed one-wayness rather than a discrete-log reduction, a strictly weaker, unproven (but standard-in-practice) assumption, and is documented here as exactly that rather than overstated. The extra wrap, `leaf = Poseidon(commitment)`, is a real, load-bearing security step and not merely an extra round: because internal Merkle nodes are a *two*-input Poseidon call (`Poseidon(left, right)`) while a leaf is a *one*-input call, and the gadget's absorption differs by input arity, leaf hashes and internal-node hashes are structurally distinct functions — functionally equivalent to the domain-separation prefix that fixes the classic Merkle second-preimage attack, where an internal node's value could otherwise be reinterpreted as a valid leaf. Given Groth16's zero-knowledge property, a verifier learns only that *some* leaf under `commitment_root` matches, never the key coordinates, the blinding factor, or the Merkle path/index — but the re-

sulting anonymity set is bounded by how many devices that identity has *actually registered* (up to the depth-8 maximum of 256 slots), not automatically 256; an identity with one or two registered devices has a correspondingly small anonymity set, stated here plainly rather than implied away by the tree's maximum depth.

6.5 Poseidon parameters

All Poseidon evaluations in the circuit and on the chain use the same fixed parameter set — BN254 scalar field, state width t=3 (rate 2, capacity 1), 8 full rounds, 57 partial rounds, x^5 S-box — gated by a canonical test vector, `Poseidon([1,2]) = 0x115cc0f5e7d690413df64c6b9662e9cf2a3617f2743245519e19607` that every implementation (the Go prover, the Cosmos chain, and — once built — the SP1 batch prover) must reproduce exactly. A single-input Poseidon call in this document means `HashTwo(x, 0)` under this same t=3 parameterization (zero-padded), not the distinct t=2 parameter set some libraries use for single-input hashing — a subtle but load-bearing distinction that has already caused cross-implementation bugs during development and is called out here deliberately.

6.6 Performance

The full circuit compiles to approximately 3,200 R1CS constraints (roughly 480 for each of the two direct Poseidon evaluations, ~1,920 across the eight Merkle-path hashes, and the remainder in range checks and the content-hash split) and proves in approximately 3–8 seconds on an iPhone 15 Pro. A prior design that additionally proved P-256 ECDSA signature verification in-circuit was deferred — emulated non-native field arithmetic for P-256 inside a BN254 circuit costs roughly 35,000 additional constraints, and the Secure Enclave / App Attest layer already provides an adequate liveness guarantee for the current threat model (§10) without paying that cost.

---

7. Verification

Verification happens at two levels. **On-chain**, `x/attestation`'s message handler reconstructs the five public inputs from the submitted message (`username_hash` recomputed from the identifier string; `content_hash_lo/hi` from the submitted content hash; `nullifier` and `commitment_root` read directly from the message) and calls the Groth16 verifier against the chain's committed verifying key. A tampered proof, a tampered public input, a forged nullifier, a stale or wrong commitment root, or a replayed nullifier are all rejected — each is covered by an explicit completeness/soundness test in the chain's test suite. **Publicly**, anyone can query `VerifyContent` by content hash without running any chain software themselves, which is what the browser verification tool and public explorer expose directly.

A second, portable verification path exists for declarations: because a declaration carries a raw device or agent signature over the content hash (rather than a ZK proof), anyone holding the signer's public key can verify authorship completely offline, without touching the chain at all — useful for archival or air-gapped verification long after the fact.

---

8. Proof Aggregation and Settlement

8.1 Why aggregate

Settling every individual attestation directly on a public L1 would be prohibitively expensive at scale. Sur Chain instead accumulates accepted attestations into a single compact commitment, and only that commitment — plus succinct inclusion proofs for anyone who needs to verify a specific attestation was included — ever touches a settlement chain.

8.2 The aggregation Merkle tree

Aggregation is implemented directly in `x/attestation` (a lighter design than the separate `x/settlement` module originally sketched for this stage — the two ended up being the same amount of code, so they were merged). Every accepted attestation contributes a leaf:

```
leaf = keccak256( keccak256(device_id) || content_hash ||
                  nullifier || keccak256(origin) )
node = keccak256( min(a,b) || max(a,b) )
       // sorted-pair, so leaf order doesn't affect the root
```

built with Ethereum's Keccak-256 (not SHA-256/Poseidon) specifically so the same construction can be verified cheaply inside an EVM contract. Two chain queries expose this: `AggregateRoot` (the current root, leaf count, and the chain height it reflects) and `MerkleProof` (an inclusion path for one attestation's content hash against that root). **Honesty note:** the current root is a running snapshot over *all* currently accepted attestations at query time, not yet chunked into discrete, closed settlement epochs — true epoch windowing (reusing the chain's existing `x/epochs` module to mark a batch closed) is designed but not yet wired up.

8.3 Ethereum settlement — implemented and tested, not yet deployed

`AttestationSettlement.sol` is written and passes 8 Foundry tests against local test infrastructure — it has **not** been deployed to Ethereum mainnet or a public testnet, and no attestation has actually been settled on Ethereum yet. The design is for a settler-gated relayer to periodically read the current `AggregateRoot` from the chain and call:

```
function submitCheckpoint(
    uint256 epochId, bytes32 root, uint256 leafCount, uint256 sourceHeight
) external;

function getCheckpoint(uint256 epochId)
    external view returns (Checkpoint memory);

function isSettled(uint256 epochId) external view returns (bool);

function verifyInclusion(
    uint256 epochId, bytes32 leaf, bytes32[] calldata proof
) external view returns (bool);
```

`verifyInclusion` reimplements the identical sorted-pair keccak folding as the chain's `merkleProof/merkleRoot` (deliberately dependency-free — the contract vendors no OpenZeppelin, only `forge-std` for tests), so that once a checkpoint is settled on Ethereum, it will be

checkable against an inclusion proof entirely on-chain, by anyone, with no further trust in Sur Chain's relayer beyond the one-time act of publishing the root itself. **Trust model, stated plainly:** the relayer is trusted to publish the *correct* root (it is designed as a "settler"-gated address, not permissionless); inclusion checking against a published root is designed to be fully trustless. Closing the remaining trust gap — making the root itself trustless, so nobody has to trust the relayer at all — is exactly what Stage 3 (§9, §12) is for; deploying this contract anywhere live is a separate, still-outstanding step on top of that (§13).

8.4 Starknet and Solana — planned

Two further settlement targets are designed but not yet built:

- **Starknet:** a Cairo contract submits the (eventual, §9) STARK proof to the Integrity verifier's fact registry [18] and records the verified epoch root — the only settlement path that stays fully post-quantum end-to-end, since Starknet verifies STARKs natively and cheaply.
- **Solana:** an Anchor program verifies a Groth16-wrapped proof via Solana's `alt_bn128` syscalls, using the `groth16-solana` library [19], and stores the epoch root in a program-derived account (PDA).

---

9. The Post-Quantum Migration Path

Sur's device proof today is Groth16 over BN254 — succinct, and the cheapest option for Ethereum verification available, but **not post-quantum**: Groth16's security rests on pairing-based / discrete-log assumptions that Shor's algorithm breaks on a sufficiently large quantum computer. Post-quantum security instead requires hash-based proof systems — STARKs, built on the FRI protocol — which are also succinct (a polylogarithmic verifier), so "succinct AND post-quantum" is achievable; it is not a trade-off Sur has to make everywhere. The trade-off that *is* real and unavoidable is narrower:

> **Post-quantum security versus cheap verification on a chain that cannot natively, cheaply verify a STARK.**

Starknet verifies STARKs natively, so nothing is sacrificed there. Ethereum and Solana cannot verify a STARK cheaply today, so **Sur wraps the STARK in a Groth16 proof** for those two chains specifically — the same approach used by SP1 [20] and RISC Zero. That wrap is the *only* place post-quantum security is given up, and only on those two chains. The post-quantum boundary therefore sits at the aggregation layer: the per-device proof stays a fast, fixed-key Groth16 proof (appropriate for a phone's compute budget); a planned Rust **SP1 batch prover** re-verifies an epoch's device proofs *inside a zkVM* and emits a genuinely post-quantum STARK, which is then wrapped in Groth16 only for the chains that need it.

This design point — STARK proving and aggregation, Groth16 wrap only where a chain demands it — is Stage 3 of the roadmap (§12) and is not yet built; the honest status today is that the per-device Groth16 layer is live and tested on Sur Chain, and the on-chain aggregation logic that would feed Ethereum settlement is implemented and tested, but the Ethereum contract itself is not deployed anywhere live (§8.3) and the *aggregate* proof root design rests on a trusted relayer rather than a recursively-verified STARK.

---

10. Security and Privacy Analysis

Completeness. Every honestly generated proof — from a registered device, with a session that satisfies the behavioral thresholds — verifies. This is enforced by automated tests on every change to the circuit or the chain's verifier.

Soundness. No proof verifies for content that was not typed on a registered device under the stated constraints, and no proof can be replayed. Both properties are tested directly: tampered proofs, tampered public inputs, forged nullifiers, wrong commitment roots, and nullifier replay are each covered by a dedicated test case in the chain's test suite (§6.2, §7).

Zero-knowledge. The proof reveals nothing about keystroke timing, the device's identity, or which of its registered commitments produced it, beyond the five public inputs listed in §6.1 — a property of the Groth16 proof system (zero-knowledge under the generic bilinear group model for its knowledge-soundness argument) and the circuit's construction, argued rather than exhaustively tested, as is standard for ZK systems (see §6.4 for the specific unlinkability argument and its anonymity-set caveat).

Hash-function assumptions. Every claim in §6.2–§6.4 that rests on Poseidon rests on a heuristic assumption — Poseidon has no standard-model collision-resistance or PRF proof the way, say, HMAC-SHA256 has under weaker assumptions. This is an accepted, documented tradeoff made for in-circuit efficiency across the ZK industry, not unique to Sur, and is stated here rather than left implicit.

Post-quantum status. Argued in detail in §9: the device proof is not post-quantum today, and the designed Ethereum settlement path (implemented, tested, not yet deployed) will not be either; the Starknet settlement path, once built, will be.

Privacy boundary. Argued in detail in §4.4: device-to-attestation linkability is intentional; device-to-person linkability is the thing the architecture avoids, with one known, disclosed residual gap (the transaction sender today is the user's own wallet).

---

11. Tokenomics

Sur Chain's native asset is **SUR** (on-chain micro-denomination `usur`; 1 SUR = 1,000,000 usur, following the standard Cosmos SDK six-decimal convention). SUR secures the network and pays for every write to the chain — device registration, attestation submission, and declarations all consume gas denominated in SUR.

11.1 Fee burn — live today

Sur Chain enforces a protocol-level fee split on every transaction, implemented as a custom ante-handler decorator (`FeeBurnDecorator`) rather than left to validator discretion:

- **80% of every transaction fee is burned** — moved to a module account with burn permission and destroyed.
- **20% is retained by the fee collector** for standard Cosmos SDK validator reward distribution.

This split is enforced identically for every transaction and cannot be changed by an individual validator; changing it requires a chain upgrade via governance. The practical effect is a usage-linked deflationary pressure: every device registration and every piece of content attested burns SUR, directly and permanently tying network activity — proving human authorship at scale — to token scarcity, rather than relying on a separate, disconnected buyback or burn mechanism.

11.2 Standard proof-of-stake security

Beyond the custom fee burn, Sur Chain runs the standard Cosmos SDK economic stack: `x/staking` for validator bonding and delegation, `x/mint` for block-reward issuance, `x/distribution` for splitting the retained 20% fee share (plus any block rewards) between validators and their delegators, and `x/slashing` for penalizing downtime or double-signing. `x/gov` allows SUR holders to vote on chain upgrades, including the fee burn ratio itself.

11.3 Community allocation

Early contributors to the network's development are recognized directly in SUR: **the project's public donation program commits to an allocation for every donating address at token generation/airdrop**, recognizing early support for the infrastructure (proof circuits, chain development, app development, and the open-source tooling around them) before the network has its own trading market. Donation addresses are published and auditable (Ethereum — which also accepts BNB/Binance Smart Chain and Base deposits at the same address — Bitcoin, Solana, and Tron).

11.4 Honest status

The `usur` denomination and the 80/20 fee-burn split are live on the current development chain today. What is **not yet finalized**, and should not be read into this document as already decided, is a mainnet genesis token allocation table, a total-supply cap, or a public token-generation-event date — those are governance and go-to-market decisions downstream of the network reaching production readiness (§13), not cryptographic or protocol facts this whitepaper can state with the same certainty as the fee-burn mechanism above.

---

12. Staged Delivery Plan and Current Status

Sur's ZK pipeline is being built and verified stage by stage, each a working, tested increment rather than a single big-bang release.

Stage	Scope	Status
1	A single device proof verifies end-to-end: fixed Groth16 proving/verifying key, real on-chain verification (replacing an earlier always-true placeholder), correct identity Merkle root maintenance, fixed 256-byte EIP-197 proof wire format	DONE (tested)
2	Epoch aggregation: accepted attestations folded into a keccak256 sorted-pair Merkle root, exposed via `AggregateRoot` / `MerkleProof`	DONE (tested; running-snapshot root, discrete epoch windowing still pending)
3	SP1 Rust batch prover: recursively verify an epoch's device proofs inside a zkVM, emit a post-quantum STARK, Groth16-wrap for EVM/Solana	PENDING
4	Settlement contracts on three chains	Ethereum: contract written & tested (8 Foundry tests), not deployed · Starknet pending · Solana pending
5	Full completeness/soundness test matrix across every layer; zero-knowledge and post-quantum analysis reconciled against the shipped implementation	PENDING (this document is part of that reconciliation)

---

13. Honest Limits

Consistent with the standard this project holds itself to elsewhere, here is what is *not yet true*, stated plainly:

- **The committed Groth16 key is a development key** from a single local trusted setup. Production use requires a multi-party trusted-setup ceremony; only the key bytes change, not the circuit or any other code.
- **The Ethereum settlement contract is not deployed anywhere live** — it exists and passes its test suite against local infrastructure only; no attestation has been settled on a real Ethereum network yet (§8.3).
- **The aggregate root design is relayer-trusted, not yet trustless.** Even once deployed,

and until the SP1 batch prover (Stage 3) lands, anyone verifying Ethereum settlement would be trusting the relayer to have published the chain's actual current root, not a recursively-verified proof of it.
- **The SP1 batch prover requires real RISC-V proving compute** and has not been built yet; it runs off-chain and is not a small undertaking.
- **Settlement contracts are validated against local/test infrastructure.** Mainnet or public-testnet deployment is a distinct, separate operational step from having a tested contract.
- **On Ethereum and Solana, the final Groth16 wrap will not be post-quantum**, by design (§9) — only the Starknet path will be, once built.
- **The transaction sender for an attestation is currently the user's own wallet account**, which is a disclosed, not-yet-closed privacy residual (§4.4).
- **`human_score`'s exact scoring formula is not yet specified in this document** (§6.3) and should be treated as an open item, not a settled design.
- **The 2000 ms IKI ceiling and the 50-point `human_score` floor are engineering choices, not literature-derived thresholds** (§6.3) — only the 20 ms IKI floor has direct support from published typing-speed data.
- **Every Poseidon-based security argument in this document is heuristic**, not reduction-based (§10) — standard practice for ZK-friendly hashes, but worth restating plainly rather than implying a stronger guarantee.
- **The Merkle anonymity set is bounded by actually-registered devices, not the tree's maximum depth** (§6.4) — an identity with few registered devices has a correspondingly small anonymity set.
- **Mainnet tokenomics (supply, allocation table, TGE timing) are not yet decided** (§11.4).

---

14. Conclusion

Sur Protocol takes a specific, falsifiable position in the fight over digital authenticity: instead of trying to detect synthetic content after the fact — a race detectors are structurally positioned to lose — it proves genuinely human content at the moment of creation, using cryptography and hardware the device already has. A zero-knowledge circuit turns typing behavior into a proof without ever revealing the behavior itself; a device identity model built from ideas about unclonable device identifiers and enclave-anchored commitment schemes keeps that proof pseudonymous by default; and a Cosmos application chain aggregates the result and is designed to settle it to public blockchains, so that no single company gets to be the arbiter of what counts as real. Not every content type can carry the same strength of evidence yet, and the system says so explicitly rather than papering over the difference. The remaining work — actually deploying the tested Ethereum settlement contract; a recursively-verified, post-quantum aggregate proof; Starknet and Solana settlement; a closed transaction-sender privacy gap; a fully specified `human_score` formula; and a fully specified mainnet token launch — is listed above without euphemism, because a provenance protocol that overstates its own guarantees would be working against the exact problem it exists to solve.

---

15. References

[1]: C2PA — Coalition for Content Provenance and Authenticity. c2pa.org
[2]: Content Authenticity Initiative — How it works. contentauthenticity.org/how-it-works
[3]: C2PA Releases Specification of World's First Industry Standard for Content Provenance. contentcredentials.org
[4]: Establishing your app's integrity — Apple Developer Documentation. developer.apple.com
[5]: Preparing to use the App Attest service — Apple Developer. developer.apple.com
[6]: Validating apps that connect to your server — Apple Developer. developer.apple.com
[7]: ZKPROV: A Zero-Knowledge Approach to Dataset Provenance for Large Language Models. arXiv:2506.20915. arxiv.org/abs/2506.20915
[8]: Zero-knowledge proof — overview and formal definitions. en.wikipedia.org/wiki/Zero-knowledge_proof
[9]: Zero-Knowledge Proofs Demystified: A Practical Code Guide for Developers. medium.com/@ancilartech
[10]: El Mathe (Eliel), "Unreplicable Device IDs." eliel.nfinic.com
[11]: El Mathe (Eliel), "Bringing Trust to the Edge: Secure Execution Without Consensus." eliel.nfinic.com
[12]: El Mathe (Eliel), "Commitment Schemes + Secure Enclave: A Powerful Combination." eliel.nfinic.com
[13]: El Mathe (Eliel), "EthSafari Discussion on ZK-Based KYC." eliel.nfinic.com
[14]: gnark — a fast zk-SNARK library written in Go (ConsenSys). github.com/Consensys/gnark
[15]: Killourhy, K. S., and Maxion, R. A. "Comparing Anomaly-Detection Algorithms for Keystroke Dynamics." IEEE/IFIP International Conference on Dependable Systems and Networks (DSN), 2009.
[16]: Dhakal, V., Feit, A. M., Kristensson, P. O., and Oulasvirta, A. "Observations on Typing from 136 Million Keystrokes." ACM CHI Conference on Human Factors in Computing Systems, 2018.
[17]: Card, S. K., Moran, T. P., and Newell, A. *The Psychology of Human-Computer Interaction*. Lawrence Erlbaum Associates, 1983. (Model Human Processor reaction-time estimates.)
[18]: Integrity (Herodotus / StarkWare) — Cairo STARK verifier on Starknet. github.com/HerodotusDev/integrity
[19]: groth16-solana (Light Protocol) — Groth16 verification via Solana `alt_bn128` syscalls. github.com/Lightprotocol/groth16-solana
[20]: SP1 (Succinct) — STARK proving, recursive compression, and Groth16/PLONK wrap for on-chain verification. docs.succinct.xyz

---

This document reflects the Sur Protocol implementation as of 2026. It is a living document, updated as the staged delivery plan (§12) progresses — see `IMPLEMENTATION_ROADMAP.md` in this repository for the engineering-level companion to this whitepaper. The behavioral-threshold and hash-security analysis in §6.3–§6.4 and §10 was independently reviewed against the actual circuit source (`surcorelibs/gnark/circuit.go`) rather than derived from this document's own prior drafts.